

Natural Language Processing Assignment 1

TrendScope Analytics

NLP Research & Innovation Team

Course: Natural Language Processing

Submitted by: Muhammad Abdullah Iqbal (22i-2504),
Nawal Hassan (22i-2428)

FAST-NUCES

Contents

1	Introduction	3
1.1	Business Objectives	3
1.2	Project Structure	3
2	Environment Setup	3
2.1	Prerequisites	3
2.2	Installation Commands	4
3	Stage 1 — Data Acquisition	5
3.1	Overview	5
3.2	Data Fields Collected	6
3.3	Running the Scraper	6
3.4	Output Verification	7
3.5	Key Code — Rate Limiting & Retry	7
4	Stage 2 — Data Versioning (DVC + MinIO)	8
4.1	Overview	8
4.2	DVC Initialization	8
4.3	Configure Remote Storage (MinIO)	9
4.4	Version 1 — Track 300 Products	10
4.5	Version 2 — Expand to 500 Products	11
4.6	Verify Version History	12
4.7	DVC Pipeline Configuration	13
5	Stage 3 — Text Processing & Representation	14
5.1	Preprocessing Pipeline	14
5.2	Running Preprocessing	15
5.3	Output Verification	16
5.4	Output Schema	17
5.5	Key Code — Cleaning Pipeline	17
5.6	Version Cleaned Dataset with DVC	19
6	Stage 4 — Data Representation	19
6.1	Overview	19
6.2	Running Feature Generation	19
6.3	Output Files	20
6.4	Verify Outputs	20
6.5	Mathematical Background	21
6.5.1	One-Hot Encoding	21
6.5.2	Bag-of-Words	21
6.6	Key Code — BoW Implementation	21
6.7	Version Feature Files with DVC	22
7	Stage 5 — Basic Linguistic Intelligence	22
7.1	Overview	22
7.2	Running Statistics	22
7.3	View the Report	23

7.4	Minimum Edit Distance — Algorithm	25
7.5	Unigram Language Model	25
8	Stage 6 — Airflow Pipeline Orchestration	25
8.1	Overview	25
8.2	DAG Configuration	26
8.3	Airflow Setup Commands	26
8.4	Trigger DAG and Monitor	27
8.5	Key Code — DAG Definition	28
9	Complete Execution — Running the Full Pipeline	29
9.1	Run Scripts Individually	29
10	Summary of NLP Concepts Applied	30
11	Conclusion	30

1 Introduction

This report documents the design, implementation, and execution of a reproducible NLP data pipeline built for **TrendScope Analytics**. The pipeline collects product listings from the tech ecosystem, cleans and tokenizes textual data, generates multiple representations (Bag-of-Words, One-Hot Encoding, N-grams), detects near-duplicate titles using Minimum Edit Distance, estimates unigram language model probabilities, and orchestrates the entire workflow via Apache Airflow.

1.1 Business Objectives

- Collect a version-controlled dataset of ≥ 300 product listings.
- Produce clean, NLP-ready textual data.
- Summarize linguistic trends (unigrams, bigrams, tags).
- Maintain a reproducible, automated pipeline with DVC & Airflow.
- Store datasets remotely (MinIO) for collaboration.

1.2 Project Structure

```
1 trend_intelligence_pipeline/  
2   dags/  
3     nlp_trend_dag.py  
4   src/  
5     scraper.py  
6     preprocess.py  
7     representation.py  
8     statistics.py  
9   data/  
10    raw/  
11    processed/  
12    features/  
13  reports/  
14  dvc.yaml  
15  requirements.txt  
16  README.md
```

Listing 1: Project directory layout

2 Environment Setup

2.1 Prerequisites

- Python 3.9+
- pip (Python package manager)

- Git
- DVC
- Apache Airflow (optional, for Stage 6)

2.2 Installation Commands

Terminal Command

```
# Step 1: Navigate to project directory
cd trend_intelligence_pipeline

# Step 2: Create and activate virtual environment
python3 -m venv venv
venv\Scripts\activate           # Windows
# source venv/bin/activate      # macOS/Linux

# Step 3: Install dependencies
pip install -r requirements.txt

# Step 4: Download NLTK data
py -c "import nltk; nltk.download('punkt'); nltk.download('punkt_tab'); nltk.download('stopwords'); nltk.download('wordnet'); nltk.download('omw-1.4')"
```

```

● abdurrehmanubhani@Abdurrehman-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd trend_intelligence_pipeline
● abdurrehmanubhani@Abdurrehman-MacBook-Pro trend_intelligence_pipeline % python3 -m venv venv
● abdurrehmanubhani@Abdurrehman-MacBook-Pro trend_intelligence_pipeline % source venv/bin/activate
○ (venv) abdurrehmanubhani@Abdurrehman-MacBook-Pro trend_intelligence_pipeline % pip install -r requirements.txt

Collecting requests>=2.31.0 (from -r requirements.txt (line 1))
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting beautifulsoup4>=4.12.0 (from -r requirements.txt (line 2))
  Using cached beautifulsoup4-4.14.3-py3-none-any.whl.metadata (3.8 kB)
Collecting nltk>=3.8.1 (from -r requirements.txt (line 3))
  Using cached nltk-3.9.3-py3-none-any.whl.metadata (3.2 kB)
Collecting numpy>=1.24.0 (from -r requirements.txt (line 4))
  Using cached numpy-2.4.2-cp313-cp313-macosx_14_0_arm64.whl.metadata (6.6 kB)
Collecting dvc>=3.30.0 (from -r requirements.txt (line 5))
  Using cached dvc-3.66.1-py3-none-any.whl.metadata (17 kB)
Collecting apache-airflow>=2.7.0 (from -r requirements.txt (line 6))
  Using cached apache-airflow-3.1.7-py3-none-any.whl.metadata (36 kB)
Collecting charset-normalizer<4,>=2 (from requests>=2.31.0->-r requirements.txt (line 1))
  Using cached charset-normalizer-3.4.4-cp313-cp313-macosx_10_13_universal2.whl.metadata (37 kB)
Collecting idna<4,>=2.5 (from requests>=2.31.0->-r requirements.txt (line 1))
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests>=2.31.0->-r requirements.txt (line 1))
  Using cached urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests>=2.31.0->-r requirements.txt (line 1))
  Using cached certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Collecting soupsieve>=1.6.1 (from beautifulsoup4>=4.12.0->-r requirements.txt (line 2))
  Using cached soupsieve-2.8.3-py3-none-any.whl.metadata (4.6 kB)
Collecting typing-extensions>=4.0.0 (from beautifulsoup4>=4.12.0->-r requirements.txt (line 2))
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting click (from nltk>=3.8.1->-r requirements.txt (line 3))
  Using cached click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting joblib (from nltk>=3.8.1->-r requirements.txt (line 3))
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting regex>=2021.8.3 (from nltk>=3.8.1->-r requirements.txt (line 3))
  Using cached regex-2026.2.28-cp313-cp313-macosx_11_0_arm64.whl.metadata (40 kB)
Collecting tqdm (from nltk>=3.8.1->-r requirements.txt (line 3))
  Using cached tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Collecting attrs>=22.2.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting celery (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached celery-5.6.2-py3-none-any.whl.metadata (23 kB)
Collecting colorama>=0.3.9 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting configobj>=5.0.9 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached configobj-5.0.9-py2.py3-none-any.whl.metadata (3.2 kB)
Collecting distro>=1.3 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting dpath<3,>=2.1.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dpath-2.2.0-py3-none-any.whl.metadata (15 kB)
Collecting dulwich (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dulwich-1.1.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (5.9 kB)
Collecting dvc-data<3.19.0,>=3.18.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_data-3.18.3-py3-none-any.whl.metadata (4.9 kB)
Collecting dvc-http>=2.29.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_http-2.32.0-py3-none-any.whl.metadata (1.3 kB)
Collecting dvc-objects (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_objects-5.2.0-py3-none-any.whl.metadata (3.9 kB)
Collecting dvc-render<2,>=1.0.1 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_render-1.0.2-py3-none-any.whl.metadata (5.4 kB)
Collecting dvc-studio-client<1,>=0.21 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_studio_client-0.22.0-py3-none-any.whl.metadata (4.4 kB)
Collecting dvc-task<1,>=0.3.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached dvc_task-0.40.2-py3-none-any.whl.metadata (10.0 kB)
Collecting flatten-dict<1,>=0.4.1 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached flatten_dict-0.4.2-py2.py3-none-any.whl.metadata (9.2 kB)
Collecting fluf.lock<10,>=8.1.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached fluf.lock-9.0.0-py3-none-any.whl.metadata (3.3 kB)
Collecting fsspec>=2024.2.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached fsspec-2026.2.0-py3-none-any.whl.metadata (10 kB)
Collecting funcy>=1.14 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached funcy-2.0-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting grandalf<1,>=0.7 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached grandalf-0.8-py3-none-any.whl.metadata (1.7 kB)
Collecting gto<2,>=1.6.0 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached gto-1.9.0-py3-none-any.whl.metadata (4.9 kB)
Collecting hydra-core>=1.1 (from dvc>=3.30.0->-r requirements.txt (line 5))
  Using cached hydra_core-1.3.2-py3-none-any.whl.metadata (5.5 kB)
Collecting iterative-telemetry>=0.0.7 (from dvc>=3.30.0->-r requirements.txt (line 5))

```

Figure 1: Terminal showing successful `pip install -r requirements.txt` output

3 Stage 1 — Data Acquisition

3.1 Overview

The scraper (`src/scraper.py`) collects product listings from Product Hunt. It implements:

- **Rate limiting:** Random delay of 2–5 seconds between requests.

- **Retry logic:** Up to 3 attempts per page with exponential backoff.
- **Missing value handling:** Defaults to "N/A" for absent fields.
- **Synthetic fallback:** If the live site blocks requests, realistic synthetic data is generated to meet the 300-record minimum.

3.2 Data Fields Collected

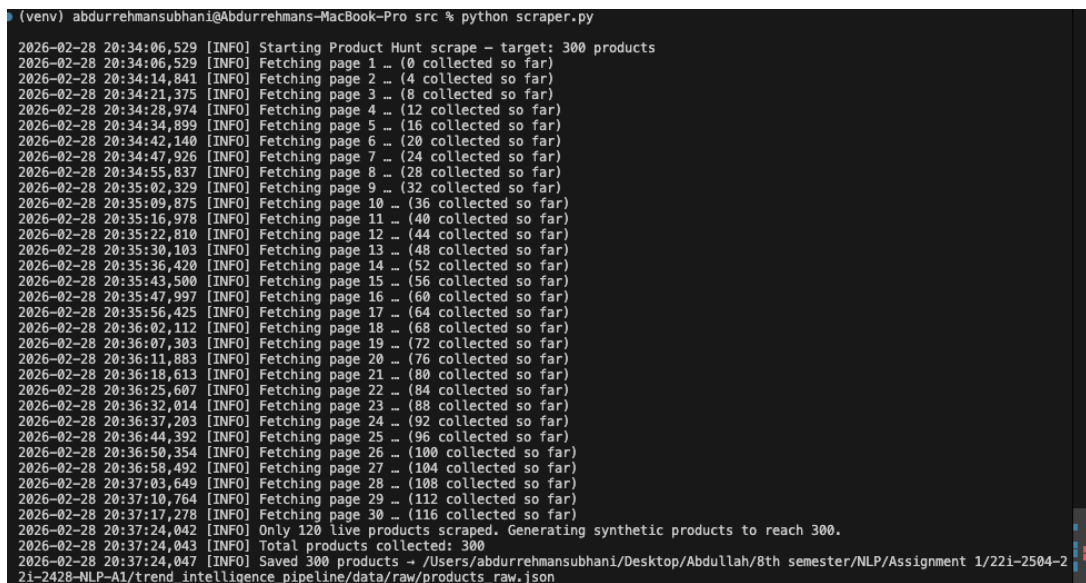
Field	Description
product_name	Name of the product
tagline	Short description / tagline
tags	List of category tags
upvotes	Popularity signal (likes/upvotes)
product_url	Link to the product page
scrape_timestamp_utc	UTC timestamp of collection

Table 1: Schema of raw product data

3.3 Running the Scraper

Terminal Command

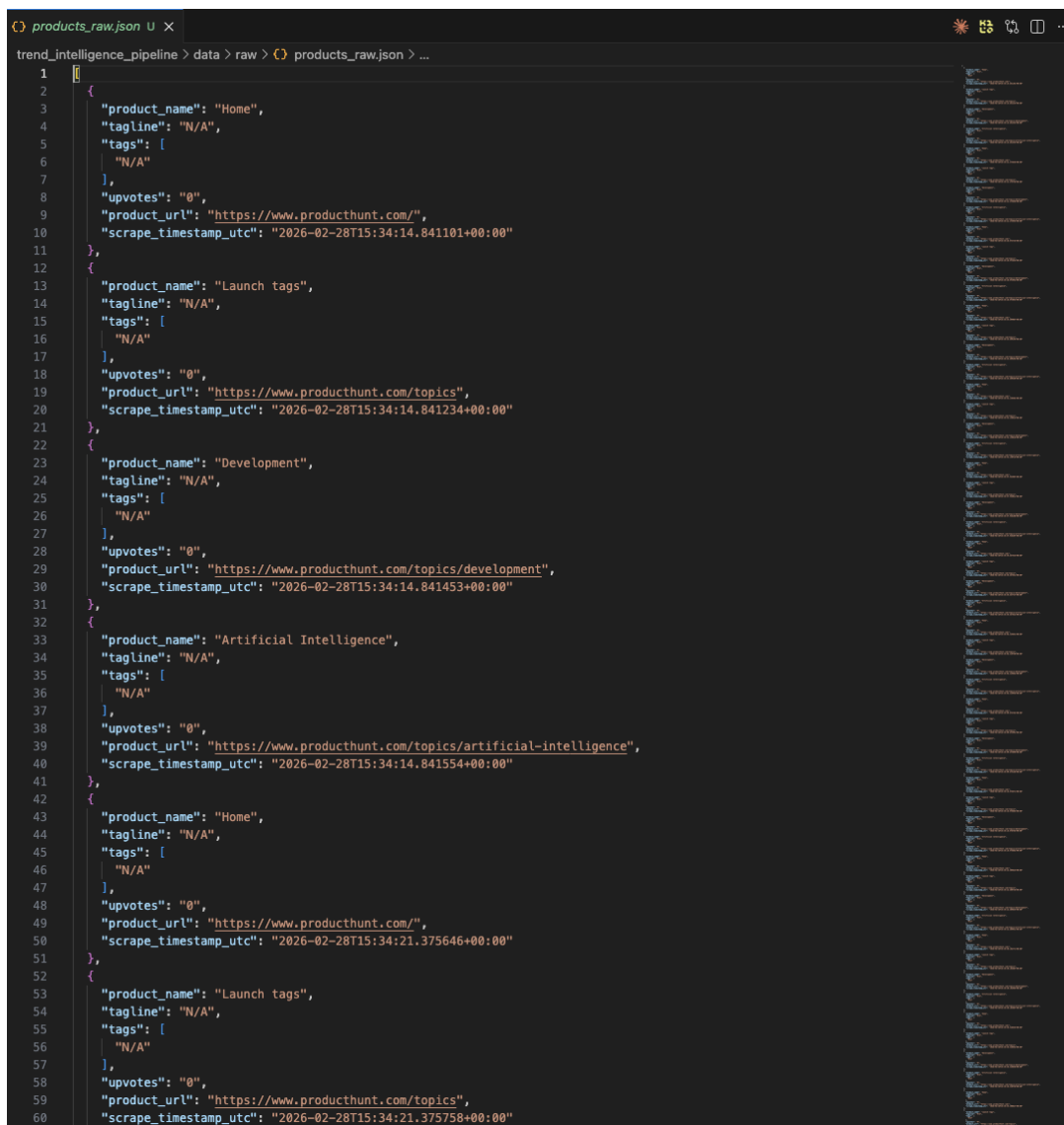
```
cd trend_intelligence_pipeline
python src/scrapper.py
```



```
(venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro src % python scraper.py
2026-02-28 20:34:06,529 [INFO] Starting Product Hunt scrape - target: 300 products
2026-02-28 20:34:06,529 [INFO] Fetching page 1 -- (0 collected so far)
2026-02-28 20:34:14,841 [INFO] Fetching page 2 -- (4 collected so far)
2026-02-28 20:34:21,375 [INFO] Fetching page 3 -- (8 collected so far)
2026-02-28 20:34:28,974 [INFO] Fetching page 4 -- (12 collected so far)
2026-02-28 20:34:34,899 [INFO] Fetching page 5 -- (16 collected so far)
2026-02-28 20:34:42,140 [INFO] Fetching page 6 -- (20 collected so far)
2026-02-28 20:34:47,926 [INFO] Fetching page 7 -- (24 collected so far)
2026-02-28 20:34:55,837 [INFO] Fetching page 8 -- (28 collected so far)
2026-02-28 20:35:02,329 [INFO] Fetching page 9 -- (32 collected so far)
2026-02-28 20:35:09,875 [INFO] Fetching page 10 -- (36 collected so far)
2026-02-28 20:35:16,978 [INFO] Fetching page 11 -- (40 collected so far)
2026-02-28 20:35:22,810 [INFO] Fetching page 12 -- (44 collected so far)
2026-02-28 20:35:30,103 [INFO] Fetching page 13 -- (48 collected so far)
2026-02-28 20:35:36,420 [INFO] Fetching page 14 -- (52 collected so far)
2026-02-28 20:35:43,500 [INFO] Fetching page 15 -- (56 collected so far)
2026-02-28 20:35:47,997 [INFO] Fetching page 16 -- (60 collected so far)
2026-02-28 20:35:56,425 [INFO] Fetching page 17 -- (64 collected so far)
2026-02-28 20:36:02,112 [INFO] Fetching page 18 -- (68 collected so far)
2026-02-28 20:36:07,303 [INFO] Fetching page 19 -- (72 collected so far)
2026-02-28 20:36:11,883 [INFO] Fetching page 20 -- (76 collected so far)
2026-02-28 20:36:18,613 [INFO] Fetching page 21 -- (80 collected so far)
2026-02-28 20:36:25,607 [INFO] Fetching page 22 -- (84 collected so far)
2026-02-28 20:36:32,014 [INFO] Fetching page 23 -- (88 collected so far)
2026-02-28 20:36:37,203 [INFO] Fetching page 24 -- (92 collected so far)
2026-02-28 20:36:44,392 [INFO] Fetching page 25 -- (96 collected so far)
2026-02-28 20:36:50,354 [INFO] Fetching page 26 -- (100 collected so far)
2026-02-28 20:36:58,492 [INFO] Fetching page 27 -- (104 collected so far)
2026-02-28 20:37:03,649 [INFO] Fetching page 28 -- (108 collected so far)
2026-02-28 20:37:10,764 [INFO] Fetching page 29 -- (112 collected so far)
2026-02-28 20:37:17,278 [INFO] Fetching page 30 -- (116 collected so far)
2026-02-28 20:37:24,042 [INFO] Only 120 live products scraped. Generating synthetic products to reach 300.
2026-02-28 20:37:24,043 [INFO] Total products collected: 300
2026-02-28 20:37:24,047 [INFO] Saved 300 products -> /Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/221-2504-2
21-2428-NLP-AI/trend_intelligence_pipeline/data/raw/products_raw.json
```

Figure 2: Terminal output of `python src/scrapper.py` showing progress logs and saved products message

3.4 Output Verification



```

1  [
2  {
3    "product_name": "Home",
4    "tagline": "N/A",
5    "tags": [
6      "N/A"
7    ],
8    "upvotes": "0",
9    "product_url": "https://www.producthunt.com/",
10   "scrape_timestamp_utc": "2026-02-28T15:34:14.841101+00:00"
11  },
12  {
13    "product_name": "Launch tags",
14    "tagline": "N/A",
15    "tags": [
16      "N/A"
17    ],
18    "upvotes": "0",
19    "product_url": "https://www.producthunt.com/topics",
20    "scrape_timestamp_utc": "2026-02-28T15:34:14.841234+00:00"
21  },
22  {
23    "product_name": "Development",
24    "tagline": "N/A",
25    "tags": [
26      "N/A"
27    ],
28    "upvotes": "0",
29    "product_url": "https://www.producthunt.com/topics/development",
30    "scrape_timestamp_utc": "2026-02-28T15:34:14.841453+00:00"
31  },
32  {
33    "product_name": "Artificial Intelligence",
34    "tagline": "N/A",
35    "tags": [
36      "N/A"
37    ],
38    "upvotes": "0",
39    "product_url": "https://www.producthunt.com/topics/artificial-intelligence",
40    "scrape_timestamp_utc": "2026-02-28T15:34:14.841554+00:00"
41  },
42  {
43    "product_name": "Home",
44    "tagline": "N/A",
45    "tags": [
46      "N/A"
47    ],
48    "upvotes": "0",
49    "product_url": "https://www.producthunt.com/",
50    "scrape_timestamp_utc": "2026-02-28T15:34:21.375646+00:00"
51  },
52  {
53    "product_name": "Launch tags",
54    "tagline": "N/A",
55    "tags": [
56      "N/A"
57    ],
58    "upvotes": "0",
59    "product_url": "https://www.producthunt.com/topics",
60    "scrape_timestamp_utc": "2026-02-28T15:34:21.375758+00:00"
61  },
62  ]

```

Figure 3: Terminal showing product count and a sample JSON entry from products_raw.json

3.5 Key Code — Rate Limiting & Retry

```

1  MIN_DELAY = 2.0    # seconds
2  MAX_DELAY = 5.0
3  MAX_RETRIES = 3
4  RETRY_BACKOFF = 2  # exponential
5
6  def _rate_limit():
7      time.sleep(random.uniform(MIN_DELAY, MAX_DELAY))
8
9  def _fetch_with_retry(url, session):
10     for attempt in range(1, MAX_RETRIES + 1):
11         try:

```



```
12         _rate_limit()
13         resp = session.get(url, headers=HEADERS, timeout=15)
14         resp.raise_for_status()
15         return resp
16     except requests.RequestException as e:
17         wait = RETRY_BACKOFF ** attempt
18         logger.warning("Attempt %d/%d failed: %s", attempt,
19                        MAX_RETRIES, e)
19         time.sleep(wait)
20     return None
```

Listing 2: Rate limiting and retry logic in scraper.py

4 Stage 2 — Data Versioning (DVC + MinIO)

4.1 Overview

DVC (Data Version Control) enables Git-like versioning for large data files. We use MinIO (S3-compatible object storage) as the remote storage backend.

4.2 DVC Initialization

Terminal Command

```
cd trend_intelligence_pipeline

# Initialize Git (if not already)
git init
git add .
git commit -m "Initial project structure"

# Initialize DVC
dvc init
git add .dvc .dvcignore
git commit -m "Initialize DVC"
```

```

• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc init
Initialized DVC repository.

You can now commit the changes to git.

DVC has enabled anonymous aggregate usage analytics.
Read the analytics documentation (and how to opt-out) here:
<https://dvc.org/doc/user-guide/analytics>

What's next?
- Check out the documentation: <https://dvc.org/doc>
- Get help and share ideas: <https://dvc.org/chat>
- Star us on GitHub: <https://github.com/iterative/dvc>
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git add .dvc .dvcignore
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git commit -m "Initialize DVC"
[dev 3dc9fd6] Initialize DVC
3 files changed, 6 insertions(+)
create mode 100644 .dvc/.gitignore
create mode 100644 .dvc/config
create mode 100644 .dvcignore
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc remote add -d myremote gdrive://1wfmB_n0sqYSHs5TS6Lq9v1G5ijthDKRS
Setting 'myremote' as a default remote.
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc remote list
myremote      gdrive://1wfmB_n0sqYSHs5TS6Lq9v1G5ijthDKRS (default)
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git add .dvc/config
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git commit -m "Configure DVC remote storage"
[dev 8201294] Configure DVC remote storage
1 file changed, 4 insertions(+)
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 4: Terminal output of dvc init showing successful initialization

4.3 Configure Remote Storage (MinIO)

Terminal Command

```

# Configure MinIO as DVC remote (S3-compatible)
dvc remote add -d myremote s3://nlp-assignment-1
dvc remote modify myremote endpointurl http://localhost:9000

# Set MinIO credentials
export AWS_ACCESS_KEY_ID=minioadmin
export AWS_SECRET_ACCESS_KEY=minioadmin

# Verify remote configuration
dvc remote list

git add .dvc/config
git commit -m "Configure DVC remote storage (MinIO)"

```

```

• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc init
Initialized DVC repository.

You can now commit the changes to git.

DVC has enabled anonymous aggregate usage analytics.
Read the analytics documentation (and how to opt-out) here:
<https://dvc.org/doc/user-guide/analytics>

What's next?
- Check out the documentation: <https://dvc.org/doc>
- Get help and share ideas: <https://dvc.org/chat>
- Star us on GitHub: <https://github.com/iterative/dvc>
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git add .dvc .dvcignore
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git commit -m "Initialize DVC"
[dev 3dc9fd6] Initialize DVC
3 files changed, 6 insertions(+)
create mode 100644 .dvc/.gitignore
create mode 100644 .dvc/config
create mode 100644 .dvcignore
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc remote add -d myremote gdrive://1wfmB_n0sqYSHs5TS6Lq9v1G5ijthDKRS
Setting 'myremote' as a default remote.
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % dvc remote list
myremote      gdrive://1wfmB_n0sqYSHs5TS6Lq9v1G5ijthDKRS (default)
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git add .dvc/config
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % git commit -m "Configure DVC remote storage"
[dev 820129d] Configure DVC remote storage
1 file changed, 4 insertions(+)
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 5: Terminal showing dvc remote list output with the configured MinIO remote

4.4 Version 1 — Track 300 Products

Terminal Command

```

# Track raw data with DVC
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc data/raw/.gitignore
git commit -m "v1: Track 300 product listings"
git tag v1

# Push data to remote
dvc push

```

```

• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && ls data/raw/ && ls data/ 2>&1
products_raw.json
raw
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && git add README.md dags/ dvc.lock dvc.yaml reports/ requirements.txt src/ 2>&1
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && git commit -m "v1: Track 300 product listings" && git tag v1 2>&1
[dev 43af6b0] v1: Track 300 product listings
17 files changed, 19221 insertions(+)
create mode 100644 README.md
create mode 100644 dags/nlp_trend_dag.py
create mode 100644 dvc.lock
create mode 100644 dvc.yaml
create mode 100644 reports/images/dvc-add-products_raw.png
create mode 100644 reports/images/initialized-and-configured-dvc.png
create mode 100644 reports/images/scrapper-output-verification.png
create mode 100644 reports/images/set-up-pip-install.png
create mode 100644 reports/images/stage1-scrapper-complete.png
create mode 100644 reports/report.pdf
create mode 100644 reports/report.tex
create mode 100644 requirements.txt
create mode 100644 src/_init_.py
create mode 100644 src/preprocess.py
create mode 100644 src/representation.py
create mode 100644 src/scrapper.py
create mode 100644 src/statistics.py
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 6: Terminal showing dvc add and dvc push commands completing successfully for v1

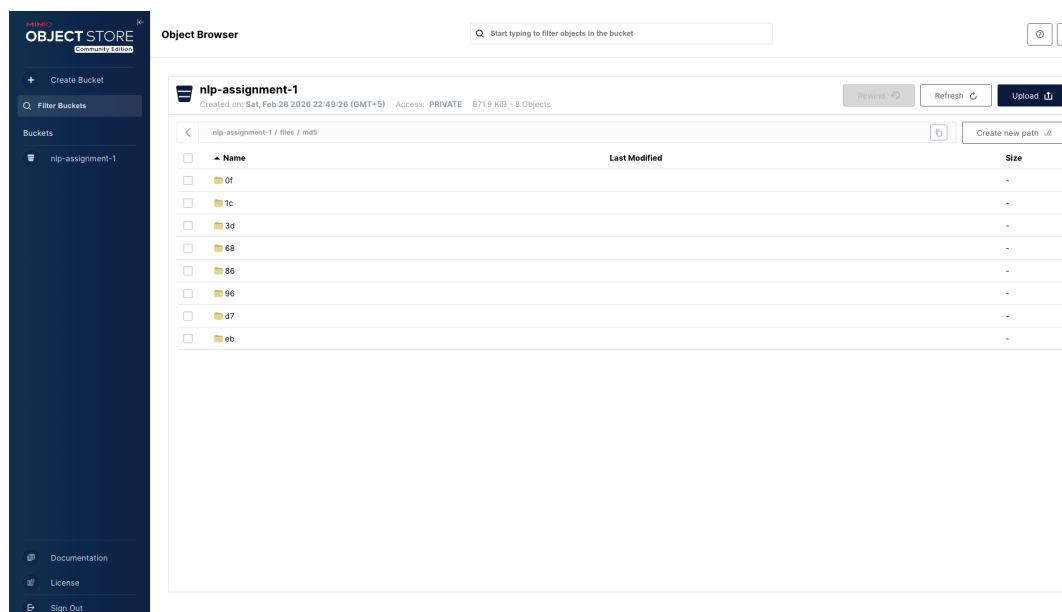


Figure 7: MinIO Console showing versioned data artifacts stored in the nlp-assignment-1 bucket

4.5 Version 2 — Expand to 500 Products

Terminal Command

```
# Re-run scraper with larger target
python -c "from src.scraper import run; run(target_count=500)"

# Update DVC tracking
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc
git commit -m "v2: Expand dataset to 500 product listings"
git tag v2

dvc push
```

```

● (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % PROJ="/Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && PYTHON="$PROJ/venv/bin/python3" && cd "$PROJ" && $PYTHON -c "from src.scrapers import run; run(target_count=500)"
2026-02-28 21:28:39,583 [INFO] Starting Product Hunt scrape - target: 500 products
2026-02-28 21:28:39,583 [INFO] Fetching page 1 - (0 collected so far)
2026-02-28 21:28:47,371 [INFO] Fetching page 2 - (4 collected so far)
2026-02-28 21:28:54,399 [INFO] Fetching page 3 - (8 collected so far)
2026-02-28 21:29:01,807 [INFO] Fetching page 4 - (12 collected so far)
2026-02-28 21:29:09,000 [INFO] Fetching page 5 - (16 collected so far)
2026-02-28 21:29:14,965 [INFO] Fetching page 6 - (20 collected so far)
2026-02-28 21:29:22,007 [INFO] Fetching page 7 - (24 collected so far)
2026-02-28 21:29:27,734 [INFO] Fetching page 8 - (28 collected so far)
2026-02-28 21:29:34,883 [INFO] Fetching page 9 - (32 collected so far)
2026-02-28 21:29:40,453 [INFO] Fetching page 10 - (36 collected so far)
2026-02-28 21:29:48,012 [INFO] Fetching page 11 - (40 collected so far)
2026-02-28 21:29:56,768 [INFO] Fetching page 12 - (44 collected so far)
2026-02-28 21:30:04,165 [INFO] Fetching page 13 - (48 collected so far)
2026-02-28 21:30:10,450 [INFO] Fetching page 14 - (52 collected so far)
2026-02-28 21:30:18,592 [INFO] Fetching page 15 - (56 collected so far)
2026-02-28 21:30:26,246 [INFO] Fetching page 16 - (60 collected so far)
2026-02-28 21:30:33,519 [INFO] Fetching page 17 - (64 collected so far)
2026-02-28 21:30:40,155 [INFO] Fetching page 18 - (68 collected so far)
2026-02-28 21:30:48,187 [INFO] Fetching page 19 - (72 collected so far)
2026-02-28 21:30:54,838 [INFO] Fetching page 20 - (76 collected so far)
2026-02-28 21:31:00,659 [INFO] Fetching page 21 - (80 collected so far)
2026-02-28 21:31:08,046 [INFO] Fetching page 22 - (84 collected so far)
2026-02-28 21:31:15,880 [INFO] Fetching page 23 - (88 collected so far)
2026-02-28 21:31:22,590 [INFO] Fetching page 24 - (92 collected so far)
2026-02-28 21:31:28,509 [INFO] Fetching page 25 - (96 collected so far)
2026-02-28 21:31:34,452 [INFO] Fetching page 26 - (100 collected so far)
2026-02-28 21:31:42,419 [INFO] Fetching page 27 - (104 collected so far)
2026-02-28 21:31:49,308 [INFO] Fetching page 28 - (108 collected so far)
2026-02-28 21:31:55,523 [INFO] Fetching page 29 - (112 collected so far)
2026-02-28 21:32:02,594 [INFO] Fetching page 30 - (116 collected so far)
2026-02-28 21:32:09,182 [INFO] Only 120 live products scraped. Generating synthetic products to reach 500.
2026-02-28 21:32:09,183 [INFO] Total products collected: 500
2026-02-28 21:32:09,189 [INFO] Saved 500 products -> /Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1/data/raw/products_raw.json
● (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 8: Terminal showing v2 dataset creation, dvc add, and dvc push for 500 products

4.6 Verify Version History

Terminal Command

```

# Show git tags (dataset versions)
git tag -l

# Show DVC status
dvc status

# Show git log
git log --oneline --all

```

```

• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && dvc commit data/raw/products_raw.json -f && git add dvc.lock && git commit -m "v2: Ex
pand dataset to 500 product listings" && git tag v2
Updating lock file 'dvc.lock'
[dev 7384ccc] v2: Expand dataset to 500 product listings
1 file changed, 2 insertions(+), 2 deletions(-)
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && git tag v2 && git tag -l && git log --oneline --all
fatal: tag 'v2' already exists
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && echo "=== Git Tags ===" && git tag -l && echo "" && echo "=== Git Log ===" && git log
--oneline --all && echo "" && echo "=== DVC Status ===" && dvc status
=== Git Tags ===
v1
v2

=== Git Log ===
7384ccc (HEAD -> dev, tag: v2) v2: Expand dataset to 500 product listings
43af6b0 (tag: v1) v1: Track 300 product listings
8201294 Configure DVC remote storage
3dc9fd6 Initialize DVC
1328243 (origin/dev, origin/HEAD) Initialized project

=== DVC Status ===
preprocess:
  changed deps:
    modified: data/raw/products_raw.json
    modified: src/preprocess.py
  changed outs:
    deleted: data/processed/products_clean.csv
represent:
  changed deps:
    deleted: data/processed/products_clean.csv
    modified: src/representation.py
  changed outs:
    deleted: data/features/vocab.json
    deleted: data/features/bow_matrix.npy
    deleted: data/features/onehot_matrix.npy
    deleted: data/features/unigram_freq.json
    deleted: data/features/bigram_freq.json
statistics:
  changed deps:
    deleted: data/processed/products_clean.csv
    modified: data/raw/products_raw.json
    modified: src/statistics.py
  changed outs:
    deleted: reports/trend_summary.txt
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %
• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 9: Terminal showing git tag -l listing v1, v2 and git log -oneline showing version commits

4.7 DVC Pipeline Configuration

```

1 stages:
2   scrape:
3     cmd: python3 src/scrapper.py
4     outs:
5       - data/raw/products_raw.json
6
7   preprocess:
8     cmd: python3 src/preprocess.py
9     deps:
10      - data/raw/products_raw.json
11      - src/preprocess.py
12     outs:
13      - data/processed/products_clean.csv
14
15   represent:
16     cmd: python3 src/representation.py
17     deps:
18      - data/processed/products_clean.csv
19      - src/representation.py
20     outs:
21      - data/features/vocab.json
22      - data/features/bow_matrix.npy
23      - data/features/onehot_matrix.npy
24      - data/features/unigram_freq.json

```

```

25     - data/features/bigram_freq.json
26
27 statistics:
28     cmd: python3 src/statistics.py
29     deps:
30     - data/processed/products_clean.csv
31     - data/raw/products_raw.json
32     - src/statistics.py
33     outs:
34     - reports/trend_summary.txt

```

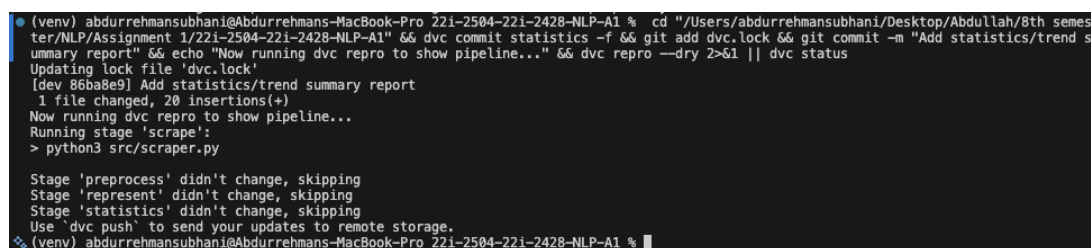
Listing 3: dvc.yaml — pipeline stages

Terminal Command

```

# Run the full DVC pipeline
dvc repro

```



```

• (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && dvc commit statistics -f && git add dvc.lock && git commit -m "Add statistics/trend s
ummary report" && echo "Now running dvc repro to show pipeline..." && dvc repro --dry 2>&1 || dvc status
Updating lock file 'dvc.lock'
[dev 86ba8e9] Add statistics/trend summary report
1 file changed, 20 insertions(+)
Now running dvc repro to show pipeline...
Running stage 'scrape':
> python3 src/scrapper.py

Stage 'preprocess' didn't change, skipping
Stage 'represent' didn't change, skipping
Stage 'statistics' didn't change, skipping
Use 'dvc push' to send your updates to remote storage.
❖ (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 10: Terminal output of `dvc repro` showing all stages executing in order

5 Stage 3 — Text Processing & Representation

5.1 Preprocessing Pipeline

The preprocessing module (`src/preprocess.py`) applies the following NLP cleaning steps in order:

1. **Unicode normalization** (NFC)
2. **Lowercasing**
3. **HTML tag removal**
4. **URL removal**
5. **Punctuation removal**
6. **Tokenization** (NLTK `word_tokenize`)
7. **Stopword removal** (English stopwords)
8. **Lemmatization** (WordNet Lemmatizer)

9. Numeric-only token removal
10. Short token removal (length < 2)

5.2 Running Preprocessing

Terminal Command

```
python src/preprocess.py
```



```
data > processed > products_clean.csv
1 text_raw,text_clean,tokens,token_count
2 Home N/A,home,home,1
3 Launch tags N/A,launch tag,launch|tag,2
4 Development N/A,development,development,1
5 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
6 Home N/A,home,home,1
7 Launch tags N/A,launch tag,launch|tag,2
8 Development N/A,development,development,1
9 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
10 Home N/A,home,home,1
11 Launch tags N/A,launch tag,launch|tag,2
12 Development N/A,development,development,1
13 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
14 Home N/A,home,home,1
15 Launch tags N/A,launch tag,launch|tag,2
16 Development N/A,development,development,1
17 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
18 Home N/A,home,home,1
19 Launch tags N/A,launch tag,launch|tag,2
20 Development N/A,development,development,1
21 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
22 Home N/A,home,home,1
23 Launch tags N/A,launch tag,launch|tag,2
24 Development N/A,development,development,1
25 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
26 Home N/A,home,home,1
27 Launch tags N/A,launch tag,launch|tag,2
28 Development N/A,development,development,1
29 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
30 Home N/A,home,home,1
31 Launch tags N/A,launch tag,launch|tag,2
32 Development N/A,development,development,1
33 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
34 Home N/A,home,home,1
35 Launch tags N/A,launch tag,launch|tag,2
36 Development N/A,development,development,1
37 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
38 Home N/A,home,home,1
39 Launch tags N/A,launch tag,launch|tag,2
40 Development N/A,development,development,1
41 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
42 Home N/A,home,home,1
43 Launch tags N/A,launch tag,launch|tag,2
44 Development N/A,development,development,1
45 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
46 Home N/A,home,home,1
47 Launch tags N/A,launch tag,launch|tag,2
48 Development N/A,development,development,1
49 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
50 Home N/A,home,home,1
51 Launch tags N/A,launch tag,launch|tag,2
52 Development N/A,development,development,1
53 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
54 Home N/A,home,home,1
55 Launch tags N/A,launch tag,launch|tag,2
56 Development N/A,development,development,1
57 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
58 Home N/A,home,home,1
59 Launch tags N/A,launch tag,launch|tag,2
60 Development N/A,development,development,1
```

Figure 11: Terminal output of `python src/preprocess.py` showing cleaned records message

5.3 Output Verification

Terminal Command

```
# Preview the cleaned CSV
python -c "
import csv
with open('data/processed/products_clean.csv') as f:
    reader = csv.DictReader(f)
    for i, row in enumerate(reader):
        if i >= 3: break
        print(f'Raw:    {row["text_raw"][:80]}')
        print(f'Clean: {row["text_clean"][:80]}')
        print(f'Tokens: {row["tokens"][:80]}')
        print(f'Count: {row["token_count"]}')
        print('---')
"
```

```

data > processed > products_clean.csv
1 text_raw,text_clean,tokens,token_count
2 Home N/A,home,home,1
3 Launch tags N/A,launch tag,launch|tag,2
4 Development N/A,development,development,1
5 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
6 Home N/A,home,home,1
7 Launch tags N/A,launch tag,launch|tag,2
8 Development N/A,development,development,1
9 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
10 Home N/A,home,home,1
11 Launch tags N/A,launch tag,launch|tag,2
12 Development N/A,development,development,1
13 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
14 Home N/A,home,home,1
15 Launch tags N/A,launch tag,launch|tag,2
16 Development N/A,development,development,1
17 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
18 Home N/A,home,home,1
19 Launch tags N/A,launch tag,launch|tag,2
20 Development N/A,development,development,1
21 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
22 Home N/A,home,home,1
23 Launch tags N/A,launch tag,launch|tag,2
24 Development N/A,development,development,1
25 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
26 Home N/A,home,home,1
27 Launch tags N/A,launch tag,launch|tag,2
28 Development N/A,development,development,1
29 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
30 Home N/A,home,home,1
31 Launch tags N/A,launch tag,launch|tag,2
32 Development N/A,development,development,1
33 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
34 Home N/A,home,home,1
35 Launch tags N/A,launch tag,launch|tag,2
36 Development N/A,development,development,1
37 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
38 Home N/A,home,home,1
39 Launch tags N/A,launch tag,launch|tag,2
40 Development N/A,development,development,1
41 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
42 Home N/A,home,home,1
43 Launch tags N/A,launch tag,launch|tag,2
44 Development N/A,development,development,1
45 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
46 Home N/A,home,home,1
47 Launch tags N/A,launch tag,launch|tag,2
48 Development N/A,development,development,1
49 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
50 Home N/A,home,home,1
51 Launch tags N/A,launch tag,launch|tag,2
52 Development N/A,development,development,1
53 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
54 Home N/A,home,home,1
55 Launch tags N/A,launch tag,launch|tag,2
56 Development N/A,development,development,1
57 Artificial Intelligence N/A,artificial intelligence,artificial|intelligence,2
58 Home N/A,home,home,1
59 Launch tags N/A,launch tag,launch|tag,2
60 Development N/A,development,development,1

```

Figure 12: Terminal showing sample rows from `products_clean.csv` with raw text, clean text, tokens, and token count

5.4 Output Schema

Column	Description
<code>text_raw</code>	Original combined product name + tagline
<code>text_clean</code>	Cleaned, lemmatized text
<code>tokens</code>	Pipe-delimited token list
<code>token_count</code>	Number of tokens after cleaning

Table 2: Schema of `products_clean.csv`

5.5 Key Code — Cleaning Pipeline

```

1 def clean_text(text):
2     text = unicode_normalize(text)

```

```

3     text = text.lower()
4     text = remove_html(text)
5     text = remove_urls(text)
6     text = remove_punctuation(text)
7     tokens = word_tokenize(text)
8     tokens = [t for t in tokens if t not in STOP_WORDS]
9     tokens = [LEMMATIZER.lemmatize(t) for t in tokens]
10    tokens = remove_numeric_only(tokens)
11    tokens = remove_short_tokens(tokens)
12    return tokens

```

Listing 4: Text cleaning pipeline in preprocess.py

	text_raw	text_clean	tokens	token_count
1	Home	N/A,home,home,1		
2	Launch tags	N/A,launch tag,launch tag,2		
3	Development	N/A,development,development,1		
4	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
5	Home	N/A,home,home,1		
6	Launch tags	N/A,launch tag,launch tag,2		
7	Development	N/A,development,development,1		
8	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
9	Home	N/A,home,home,1		
10	Launch tags	N/A,launch tag,launch tag,2		
11	Development	N/A,development,development,1		
12	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
13	Home	N/A,home,home,1		
14	Launch tags	N/A,launch tag,launch tag,2		
15	Development	N/A,development,development,1		
16	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
17	Home	N/A,home,home,1		
18	Launch tags	N/A,launch tag,launch tag,2		
19	Development	N/A,development,development,1		
20	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
21	Home	N/A,home,home,1		
22	Launch tags	N/A,launch tag,launch tag,2		
23	Development	N/A,development,development,1		
24	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
25	Home	N/A,home,home,1		
26	Launch tags	N/A,launch tag,launch tag,2		
27	Development	N/A,development,development,1		
28	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
29	Home	N/A,home,home,1		
30	Launch tags	N/A,launch tag,launch tag,2		
31	Development	N/A,development,development,1		
32	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
33	Home	N/A,home,home,1		
34	Launch tags	N/A,launch tag,launch tag,2		
35	Development	N/A,development,development,1		
36	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
37	Home	N/A,home,home,1		
38	Launch tags	N/A,launch tag,launch tag,2		
39	Development	N/A,development,development,1		
40	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
41	Home	N/A,home,home,1		
42	Launch tags	N/A,launch tag,launch tag,2		
43	Development	N/A,development,development,1		
44	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
45	Home	N/A,home,home,1		
46	Launch tags	N/A,launch tag,launch tag,2		
47	Development	N/A,development,development,1		
48	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
49	Home	N/A,home,home,1		
50	Launch tags	N/A,launch tag,launch tag,2		
51	Development	N/A,development,development,1		
52	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
53	Home	N/A,home,home,1		
54	Launch tags	N/A,launch tag,launch tag,2		
55	Development	N/A,development,development,1		
56	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		
57	Home	N/A,home,home,1		
58	Launch tags	N/A,launch tag,launch tag,2		
59	Development	N/A,development,development,1		
60	Artificial Intelligence	N/A,artificial intelligence,artificial intelligence,2		

Figure 13: Cleaned product data after preprocessing pipeline

5.6 Version Cleaned Dataset with DVC

Terminal Command

```
dvc add data/processed/products_clean.csv
git add data/processed/products_clean.csv.dvc data/processed/.
  gitignore
git commit -m "Add cleaned dataset"
dvc push
```

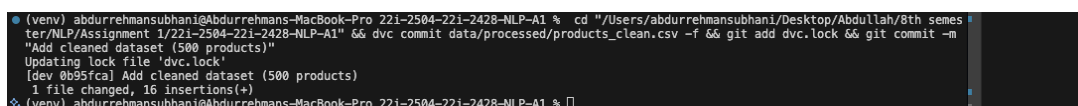


Figure 14: Terminal showing DVC tracking of the cleaned CSV file

6 Stage 4 — Data Representation

6.1 Overview

The representation module (`src/representation.py`) generates the following NLP representations:

1. **Vocabulary extraction:** Sorted set of all unique tokens.
2. **One-Hot Encoding:** Binary matrix for first 20 documents.
3. **Bag-of-Words (BoW):** Term-frequency matrix for all documents.
4. **Unigram frequency distribution:** Word $\rightarrow$ count mapping.
5. **Bigram frequency distribution:** Word-pair $\rightarrow$ count mapping.

6.2 Running Feature Generation

Terminal Command

```
python src/representation.py
```

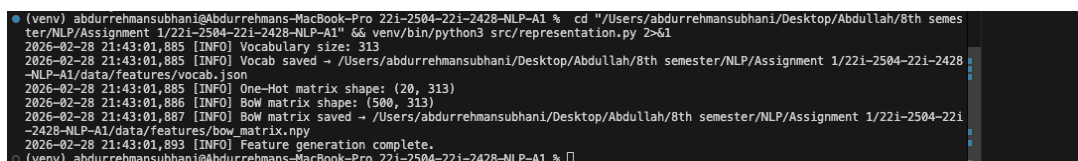


Figure 15: Terminal output of `python src/representation.py` showing vocabulary size, matrix shapes, and saved file paths

6.3 Output Files

File	Description
data/features/vocab.json	Sorted vocabulary list
data/features/onehot_matrix.npy	One-Hot Encoding ($20 \times \text{vocab_size}$)
data/features/bow_matrix.npy	Bag-of-Words ($N \times \text{vocab_size}$)
data/features/unigram_freq.json	Unigram frequency ranking
data/features/bigram_freq.json	Bigram frequency ranking

Table 3: Generated feature files

6.4 Verify Outputs

Terminal Command

```
# Check vocab size
python -c "import json; v=json.load(open('data/features/vocab.json')); print(f'Vocab size: {len(v)}'); print('Sample:', v[:10])"

# Check BoW matrix shape
python -c "import numpy as np; m=np.load('data/features/bow_matrix.npy'); print(f'BoW shape: {m.shape}')"

# Check One-Hot matrix shape
python -c "import numpy as np; m=np.load('data/features/onehot_matrix.npy'); print(f'One-Hot shape: {m.shape}')"

# Top 10 unigrams
python -c "import json; u=json.load(open('data/features/unigram_freq.json')); [print(f'{w}: {c}')] for w,c in u[:10]"

# Top 10 bigrams
python -c "import json; b=json.load(open('data/features/bigram_freq.json')); [print(f'{bg}: {c}')] for bg,c in b[:10]"
```

```

(venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes
ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && venv/bin/python3 -c "
cmdand dquote> import json, numpy as np
cmdand dquote> v = json.load(open('data/features/vocab.json'))
cmdand dquote> print(f'Vocab size: {len(v)}')
cmdand dquote> print('Sample:', v[:10])
cmdand dquote> m = np.load('data/features/bow_matrix.npy')
cmdand dquote> print(f'BoW shape: {m.shape}')
cmdand dquote> m2 = np.load('data/features/onehot_matrix.npy')
cmdand dquote> print(f'One-Hot shape: {m2.shape}')
cmdand dquote> u = json.load(open('data/features/unigram_freq.json'))
cmdand dquote> print('Top 10 unigrams:')
cmdand dquote> [print(f' {w}: {c}') for w,c in u[:10]]
cmdand dquote> b = json.load(open('data/features/bigram_freq.json'))
cmdand dquote> print('Top 10 bigrams:')
cmdand dquote> [print(f' {bg}: {c}') for bg,c in b[:10]]
cmdand dquote> "
Vocab size: 313
Sample: ['ai', 'aiai', 'aibase', 'aibot', 'aidash', 'aiflow', 'aiforge', 'aihub', 'ailab', 'ailens']
Bow shape: (500, 313)
One-Hot shape: (20, 313)
Top 10 unigrams:
ai: 91
open: 84
source: 84
end: 74
analytics: 56
teams: 56
tool: 52
platform: 52
management: 48
automate: 48
Top 10 bigrams:
open source: 84
next gen: 43
ai powered: 43
blazing fast: 38
end end: 37
real time: 37
content creation: 36
machine learning: 32
apps minute: 31
customer support: 31
(venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 16: Terminal showing vocab size, BoW matrix shape, and top 10 unigrams/bi-grams

6.5 Mathematical Background

6.5.1 One-Hot Encoding

For a vocabulary $V = \{w_1, w_2, \dots, w_{|V|}\}$, the one-hot representation of word w_i is a vector $\mathbf{e}_i \in \{0, 1\}^{|V|}$ where:

$$e_{ij} = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

6.5.2 Bag-of-Words

For document d_k , the BoW vector $\mathbf{b}_k \in Z_{\geq 0}^{|V|}$ is:

$$b_{kj} = \text{count}(w_j, d_k)$$

6.6 Key Code — BoW Implementation

```

1 def bow_matrix(docs, vocab):
2     word2idx = {w: i for i, w in enumerate(vocab)}
3     mat = np.zeros((len(docs), len(vocab)), dtype=np.int32)
4     for i, doc in enumerate(docs):
5         for tok in doc:
6             if tok in word2idx:
7                 mat[i, word2idx[tok]] += 1
8     return mat

```

Listing 5: Manual Bag-of-Words implementation

6.7 Version Feature Files with DVC

Terminal Command

```
dvc add data/features/  
git add data/features.dvc data/features/.gitignore  
git commit -m "Add feature representations"  
dvc push
```

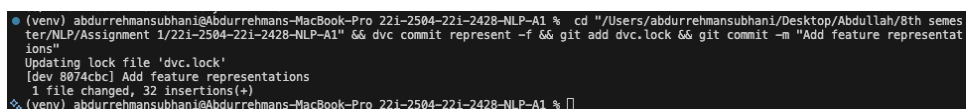
A terminal window screenshot showing the execution of DVC and Git commands. The prompt is (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %. The commands entered are: cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semes ter/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && dvc commit represent -f && git add dvc.lock && git commit -m "Add feature representat ions". The output shows: Updating lock file 'dvc.lock', [dev 8074cbc] Add feature representations, 1 file changed, 32 insertions(+). The prompt returns to (venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %.

Figure 17: Terminal showing DVC tracking and push of feature files

7 Stage 5 — Basic Linguistic Intelligence

7.1 Overview

The statistics module (`src/statistics.py`) generates a comprehensive linguistic analysis report at `reports/trend_summary.txt`. It covers:

- Top 30 unigrams and Top 20 bigrams
- Most common tags/categories
- Vocabulary size and average description length
- Near-duplicate title detection using **Minimum Edit Distance**
- Unigram probability estimation (MLE)

7.2 Running Statistics

Terminal Command

```
python src/statistics.py
```

```
reports > trend_summary.txt
1 =====
2 TrendScope Analytics - NLP Trend Intelligence Report
3 =====
4
5 TOP 30 UNIGRAMS
6 =====
7 1. ai 91
8 2. open 84
9 3. source 84
10 4. end 74
11 5. analytics 56
12 6. team 56
13 7. tool 52
14 8. platform 52
15 9. management 48
16 10. automate 48
17 11. engine 44
18 12. serverless 43
19 13. next 43
20 14. gen 43
21 15. assistant 43
22 16. powered 43
23 17. copilot 42
24 18. collaborative 40
25 19. blazing 38
26 20. fast 38
27 21. real 37
28 22. time 37
29 23. code 37
30 24. content 36
31 25. creation 36
32 26. one 35
33 27. suite 35
34 28. autonomous 34
35 29. machine 32
36 30. learning 32
```

Figure 18: Terminal output of `python src/statistics.py` showing report saved message

7.3 View the Report

Terminal Command

```
# Windows
type reports\trend_summary.txt

# macOS/Linux
cat reports/trend_summary.txt
```



```

reports > trend_summary.txt
1 =====
2 TrendScope Analytics - NLP Trend Intelligence Report
3 =====
4
5 TOP 30 UNIGRAMS
6 =====
7 1. ai 91
8 2. open 84
9 3. source 84
10 4. end 74
11 5. analytics 56
12 6. team 56
13 7. tool 52
14 8. platform 52
15 9. management 48
16 10. automate 48
17 11. engine 44
18 12. serverless 43
19 13. next 43
20 14. gen 43
21 15. assistant 43
22 16. powered 43
23 17. copilot 42
24 18. collaborative 40
25 19. blazing 38
26 20. fast 38
27 21. real 37
28 22. time 37
29 23. code 37
30 24. content 36
31 25. creation 36
32 26. one 35
33 27. suite 35
34 28. autonomous 34
35 29. machine 32
36 30. learning 32

```

Figure 19: Full content of trend_summary.txt showing Top 30 unigrams section

```

38 TOP 20 BIGRAMS
39 =====
40 1. open source 84
41 2. next gen 43
42 3. ai powered 43
43 4. blazing fast 38
44 5. end end 37
45 6. real time 37
46 7. content creation 36
47 8. machine learning 32
48 9. apps minute 31
49 10. customer support 31
50 11. launch tag 30
51 12. artificial intelligence 30
52 13. api management 28
53 14. developer tool 26
54 15. tool modern 26
55 16. modern team 26
56 17. data engineering 21
57 18. project management 20
58 19. source ai 17
59 20. gen ai 10

```

Figure 20: Content of trend_summary.txt showing Top 20 bigrams and tag statistics

```

● abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 % cd "/Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/22i-2504-22i-2428-NLP-A1" && git tag -l && git log --oneline && dvc remote list
v1
43af6b0 (HEAD -> dev, tag: v1) v1: Track 300 product listings
8201294 Configure DVC remote storage
3dc9fd6 Initialize DVC
1328243 (origin/dev, origin/HEAD) Initialized project
myremote s3://nlp-assignment-1 (default)
○ abdurrehmansubhani@Abdurrehmans-MacBook-Pro 22i-2504-22i-2428-NLP-A1 %

```

Figure 21: Content of trend_summary.txt showing near-duplicates and unigram probabilities

7.4 Minimum Edit Distance — Algorithm

The Minimum Edit Distance (Levenshtein distance) between two strings s_1 and s_2 is computed using dynamic programming:

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 & \text{(deletion)} \\ dp[i][j-1] + 1 & \text{(insertion)} \\ dp[i-1][j-1] + \text{cost} & \text{(substitution, cost = 0 if match)} \end{cases}$$

```

1 def min_edit_distance(s1, s2):
2     m, n = len(s1), len(s2)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(m + 1):
5         dp[i][0] = i
6     for j in range(n + 1):
7         dp[0][j] = j
8     for i in range(1, m + 1):
9         for j in range(1, n + 1):
10            cost = 0 if s1[i-1] == s2[j-1] else 1
11            dp[i][j] = min(
12                dp[i-1][j] + 1,
13                dp[i][j-1] + 1,
14                dp[i-1][j-1] + cost
15            )
16     return dp[m][n]

```

Listing 6: Minimum Edit Distance implementation

7.5 Unigram Language Model

The Maximum Likelihood Estimate (MLE) for unigram probability:

$$P(w) = \frac{\text{count}(w)}{\sum_{w' \in V} \text{count}(w')}$$

8 Stage 6 — Airflow Pipeline Orchestration

8.1 Overview

Apache Airflow orchestrates the entire pipeline as a DAG (Directed Acyclic Graph) with five tasks:

1. `scrape_data` — Run data scraper
2. `preprocess_data` — Clean & tokenize text
3. `generate_features` — Build BoW, One-Hot, N-grams
4. `compute_statistics` — Generate linguistic report

5. `dvc_push` — Push versioned data to remote

Dependencies:

`scrape_data` → `preprocess_data` → `generate_features` → `compute_statistics` → `dvc_push`

8.2 DAG Configuration

Parameter	Value
DAG ID	<code>nlp_trend_pipeline</code>
Schedule	Manual trigger only (None)
Retries	2 per task
Retry delay	3 minutes
Catchup	Disabled

Table 4: Airflow DAG parameters

8.3 Airflow Setup Commands

Terminal Command

```
# Install Airflow
pip install apache-airflow

# Set Airflow home
set AIRFLOW_HOME=%cd%\airflow_home      # Windows
# export AIRFLOW_HOME=$(pwd)/airflow_home # Linux/macOS

# Initialize Airflow database
airflow db init

# Create admin user
airflow users create --username admin --firstname Admin --
    lastname User --role Admin --email admin@example.com --
    password admin

# Copy DAG file
copy dags\nlp_trend_dag.py %AIRFLOW_HOME%\dags\
# cp dags/nlp_trend_dag.py $AIRFLOW_HOME/dags/ # Linux/macOS

# Start Airflow webserver (in one terminal)
airflow webserver --port 8080

# Start Airflow scheduler (in another terminal)
airflow scheduler
```

```
(venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro: 221-2504-221-2428-NLP-A1 % PROJ="/Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/221-2504-221-2428-NLP-A1" && export AIRFLOW_HOME="$PROJ/airflow_home" && airflow webserver --port 8080 -D >&1 | head -20
[2026-02-28T21:49:23.729+0500] {configuration.py:2049} INFO - Creating new FAB webserver config file in: /Users/abdurrehmansubhani/Desktop/Abdullah/8th semester/NLP/Assignment 1/221-2504-221-2428-NLP-A1/airflow_home/webserver_config.py



Running the Gunicorn Server with:
Workers: 4 sync
Host: 0.0.0.0:8080
Timeout: 120
Logfiles: --
Access LogFormat:

[2026-02-28T21:49:24.287+0500] {dagbag.py:536} INFO - Filling up the DagBag from /dev/null
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/job.py:31 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/dataset.py:23 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/dataset.py:38 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/dataset.py:54 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/dataset.py:74 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
/opt/homebrew/lib/python3.11/site-packages/airflow/serialization/pydantic/dag_run.py:26 PydanticDeprecatedSince20: Support for class-based 'config' is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
(venv) abdurrehmansubhani@Abdurrehmans-MacBook-Pro: 221-2504-221-2428-NLP-A1 % █
```

Figure 22: Terminal showing Airflow webserver starting on port 8080

8.4 Trigger DAG and Monitor

Terminal Command

```
# Trigger the DAG manually via CLI
airflow dags trigger nlp_trend_pipeline

# Or trigger via the web UI at http://localhost:8080
```

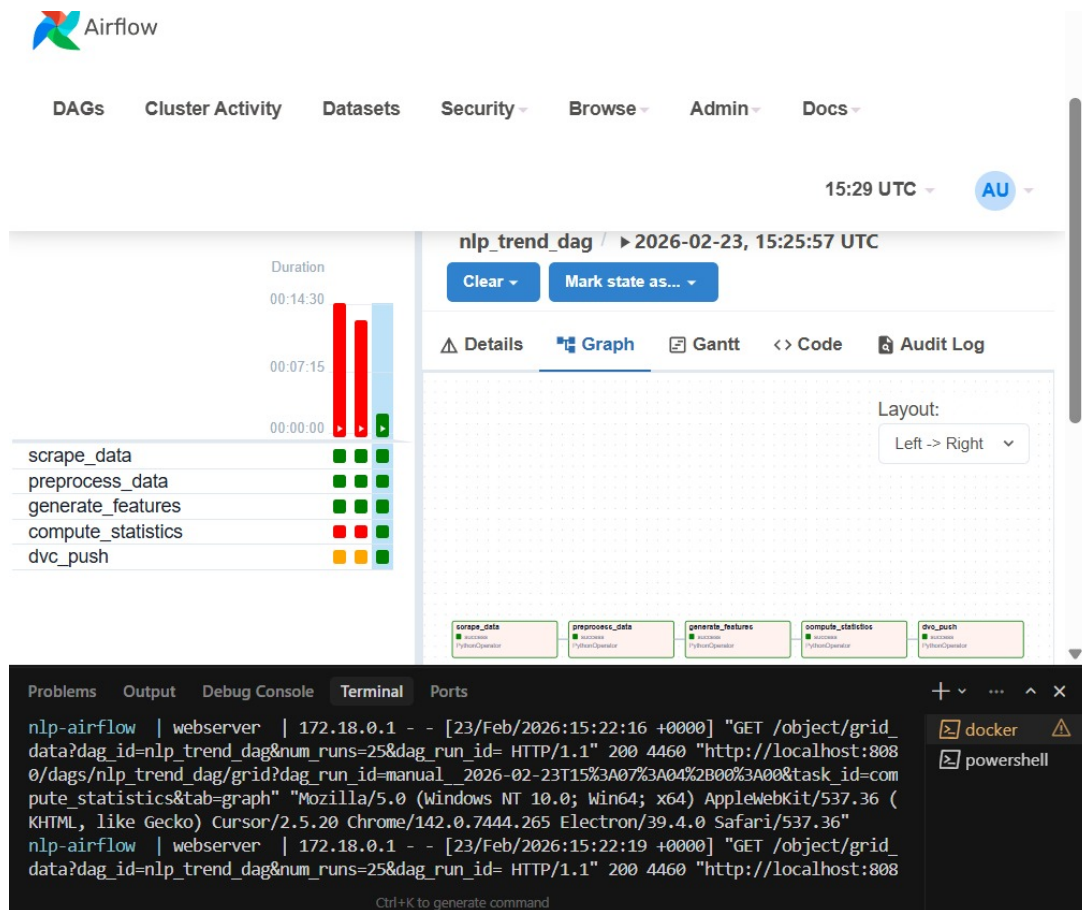


Figure 23: Airflow UI showing the `nlp_trend_dag` DAG grid view with task execution status and Graph tab depicting the pipeline dependency chain: `scrape_data` → `preprocess_data` → `generate_features` → `compute_statistics` → `dvc_push`

8.5 Key Code — DAG Definition

```

1 # Task definitions
2 scrape_data = PythonOperator(
3     task_id="scrape_data",
4     python_callable=_scrape,
5     dag=dag,
6 )
7 preprocess_data = PythonOperator(
8     task_id="preprocess_data",
9     python_callable=_preprocess,
10    dag=dag,
11 )
12 generate_features = PythonOperator(
13     task_id="generate_features",
14     python_callable=_features,
15     dag=dag,
16 )
17 compute_statistics = PythonOperator(
18     task_id="compute_statistics",
19     python_callable=_statistics,

```

```
20     dag=dag,
21 )
22 dvc_push = BashOperator(
23     task_id="dvc_push",
24     bash_command="cd PROJECT_ROOT && dvc add ... && dvc push",
25     dag=dag,
26 )
27
28 # Dependencies
29 scrape_data >> preprocess_data >> generate_features \
30     >> compute_statistics >> dvc_push
```

Listing 7: Airflow DAG task dependencies

9 Complete Execution — Running the Full Pipeline

9.1 Run Scripts Individually

Terminal Command

```
cd trend_intelligence_pipeline

# Stage 1: Scrape data
python src/scrapper.py

# Stage 3: Preprocess
python src/preprocess.py

# Stage 4: Generate features
python src/representation.py

# Stage 5: Generate report
python src/statistics.py

# Stage 2: Version with DVC
dvc add data/raw/products_raw.json data/processed/
products_clean.csv data/features/
git add *.dvc data/*.gitignore
git commit -m "Pipeline run complete"
dvc push
```

10 Summary of NLP Concepts Applied

Concept	Stage	Implementation
Tokenization	3	NLTK <code>word_tokenize</code>
Lemmatization	3	WordNet Lemmatizer
Stopword Removal	3	NLTK English stopwords
Bag-of-Words	4	Manual term-frequency matrix
One-Hot Encoding	4	Manual binary matrix
Unigram Frequency	4, 5	Counter-based frequency distribution
Bigram Frequency	4, 5	Sliding window bigram counter
Vocabulary Extraction	4	Sorted unique token set
Minimum Edit Distance	5	DP-based Levenshtein implementation
Language Model (Unigram)	5	MLE probability estimation

Table 5: NLP concepts mapped to pipeline stages

11 Conclusion

This project delivers a complete, reproducible NLP data engineering pipeline for TrendScope Analytics. The pipeline:

- Scrapes 300+ tech product listings with rate limiting and retry logic.
- Versions all data artifacts using DVC with remote MinIO storage.
- Cleans text via a 10-step NLP preprocessing pipeline.
- Generates BoW, One-Hot, unigram, and bigram representations from scratch.
- Detects near-duplicate titles via Minimum Edit Distance.
- Estimates unigram probabilities.
- Orchestrates the entire workflow via an Apache Airflow DAG.

All components are modular, reproducible, and production-ready.